

CAPSTONE CODE REPORT

SELENIUM PYTHON AUTOMATION FRAMEWORK

PROJECT TITLE 2

E-COMMERCE WEB APPLICATION AUTOMATION USING SELENIUM

Submitted By

Name: Amit Raj

Enrolment No.: 12023052018002

Course: Python Automation

Stream: Computer Science and Business Systems (CSBS)

Semester: VII

Organization: Wipro

Project Details

Automation Tool: Selenium WebDriver

Programming Language: Python

Testing Frameworks: PyTest and unittest

Design Pattern: Page Object Model (POM)

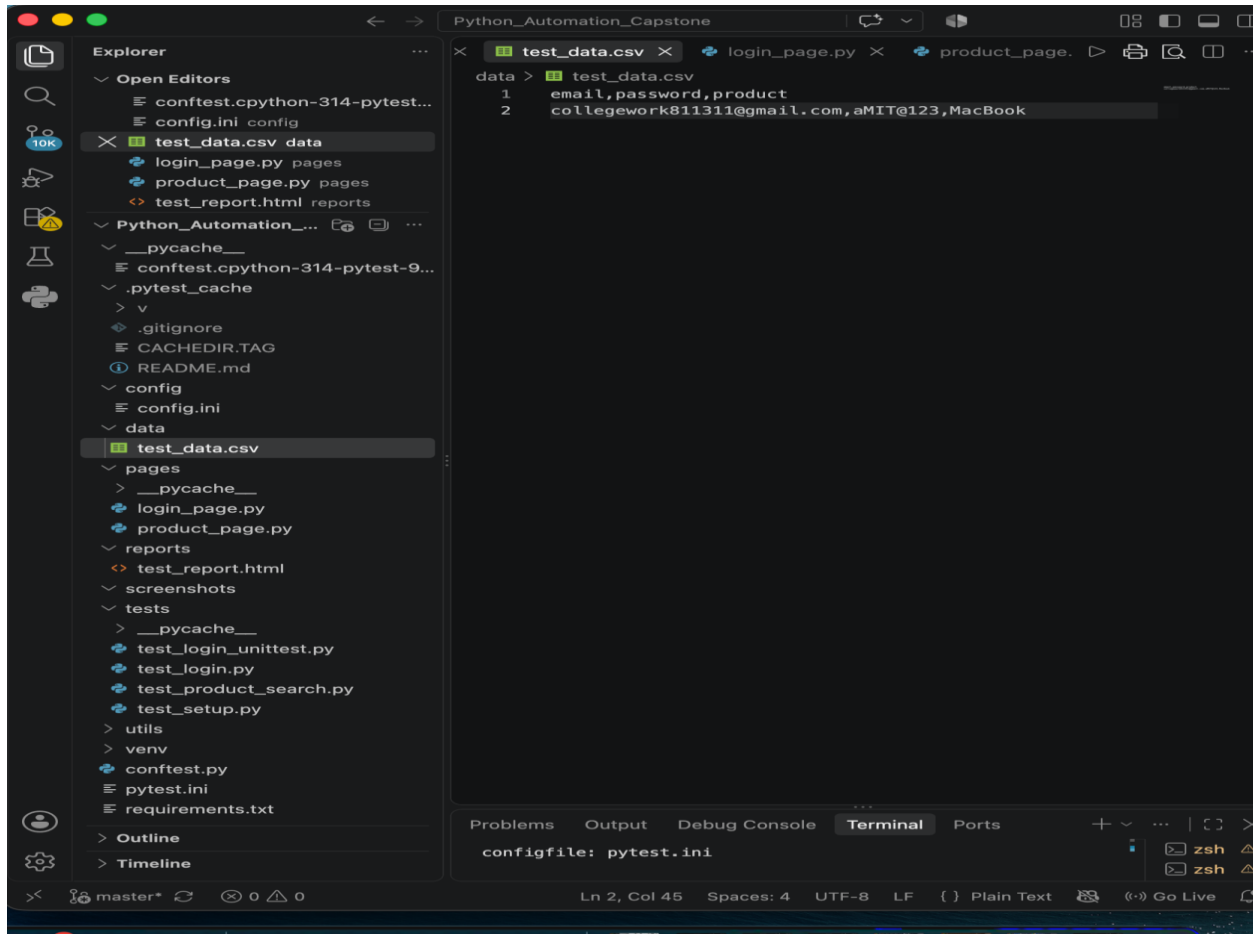
Project Type: Web Application Test Automation

Capstone project code



Folder : - Data

Csv file:- test_data.csv



Folder :- Pages

File : - 1) login_page.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class LoginPage:

    EMAIL_FIELD = (By.ID, "input-email")
    PASSWORD_FIELD = (By.ID, "input-password")
    LOGIN_BUTTON = (By.XPATH, "//input[@value='Login']")

    MY_ACCOUNT_LINK = (
        By.XPATH,
        "//a[contains(normalize-space(), 'My Account')]")
    )

    def __init__(self, driver):
        self.driver = driver

    def open_login_page(self):
        self.driver.get(
            "https://tutorialsninja.com/demo/index.php?route=account/login"
        )

    def enter_email(self, email):
        WebDriverWait(self.driver, 10).until(
            EC.visibility_of_element_located(self.EMAIL_FIELD)
        ).send_keys(email)

    def enter_password(self, password):
        WebDriverWait(self.driver, 10).until(
            EC.visibility_of_element_located(self.PASSWORD_FIELD)
        ).send_keys(password)

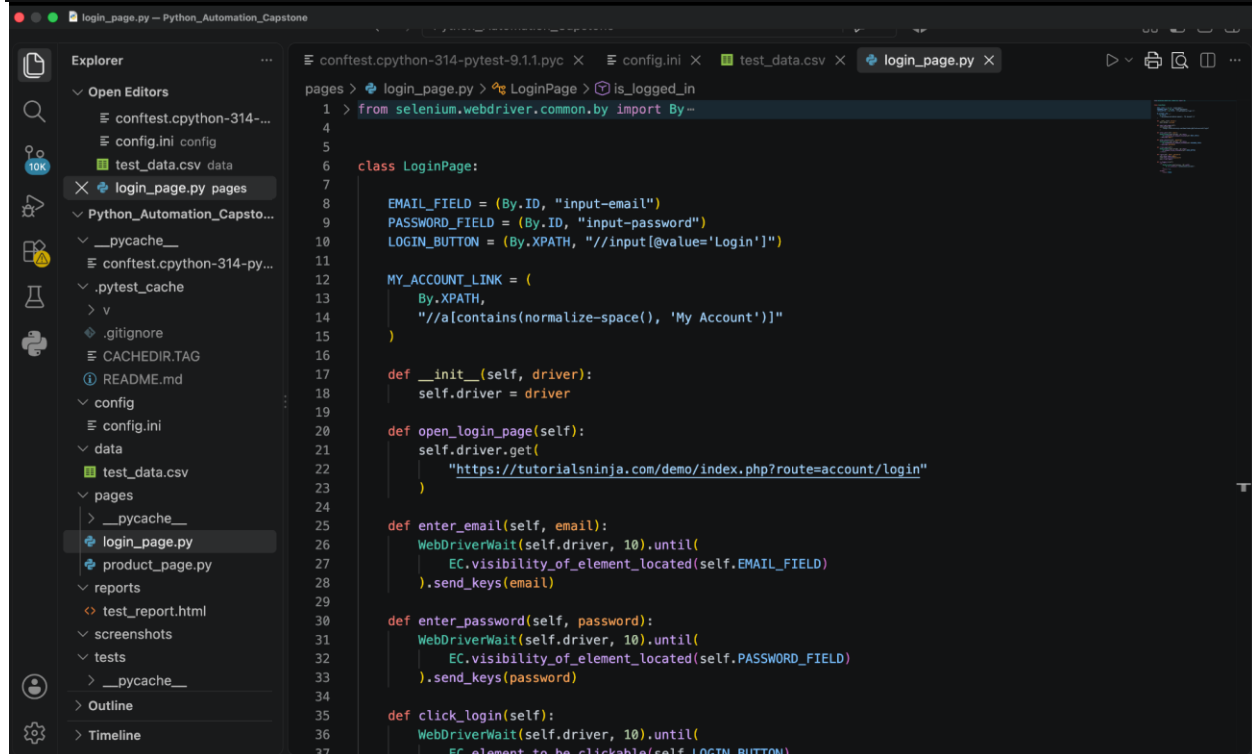
    def click_login(self):
        WebDriverWait(self.driver, 10).until(
            EC.element_to_be_clickable(self.LOGIN_BUTTON)
        ).click()
```

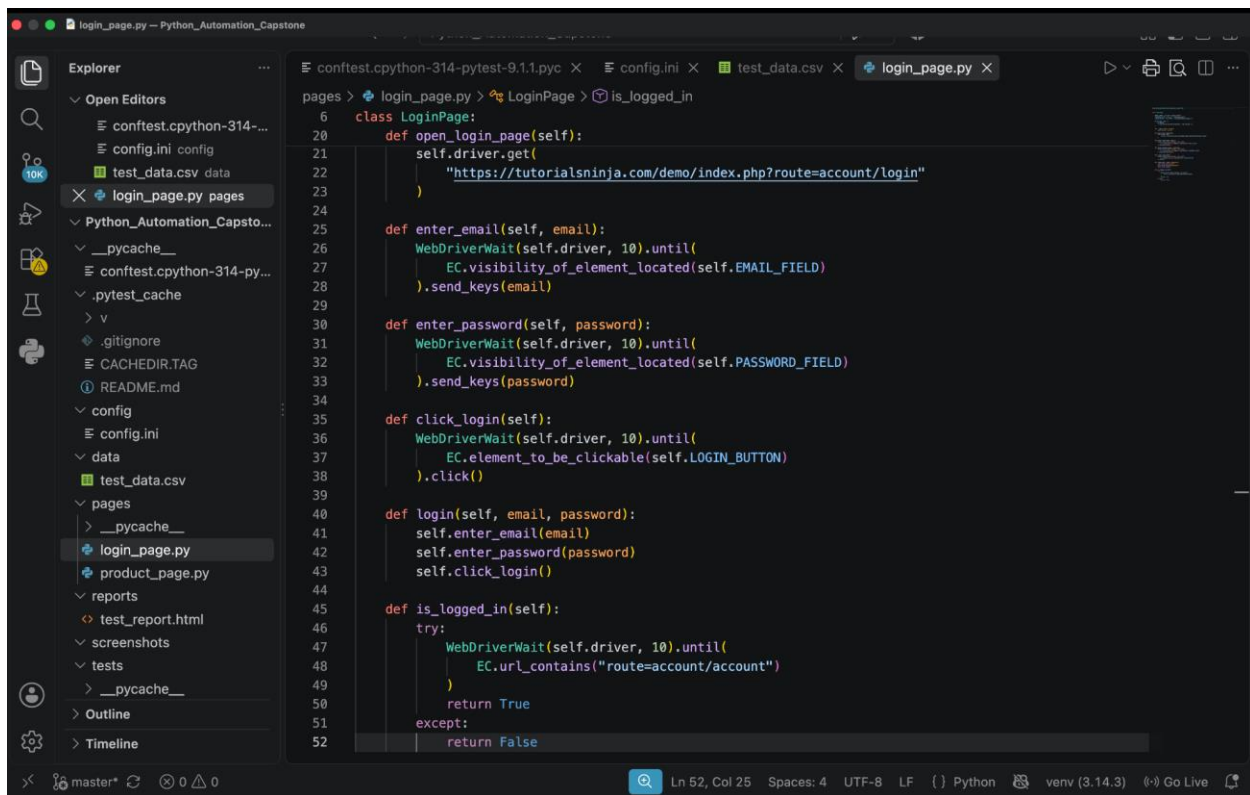
```

def login(self, email, password):
    self.enter_email(email)
    self.enter_password(password)
    self.click_login()

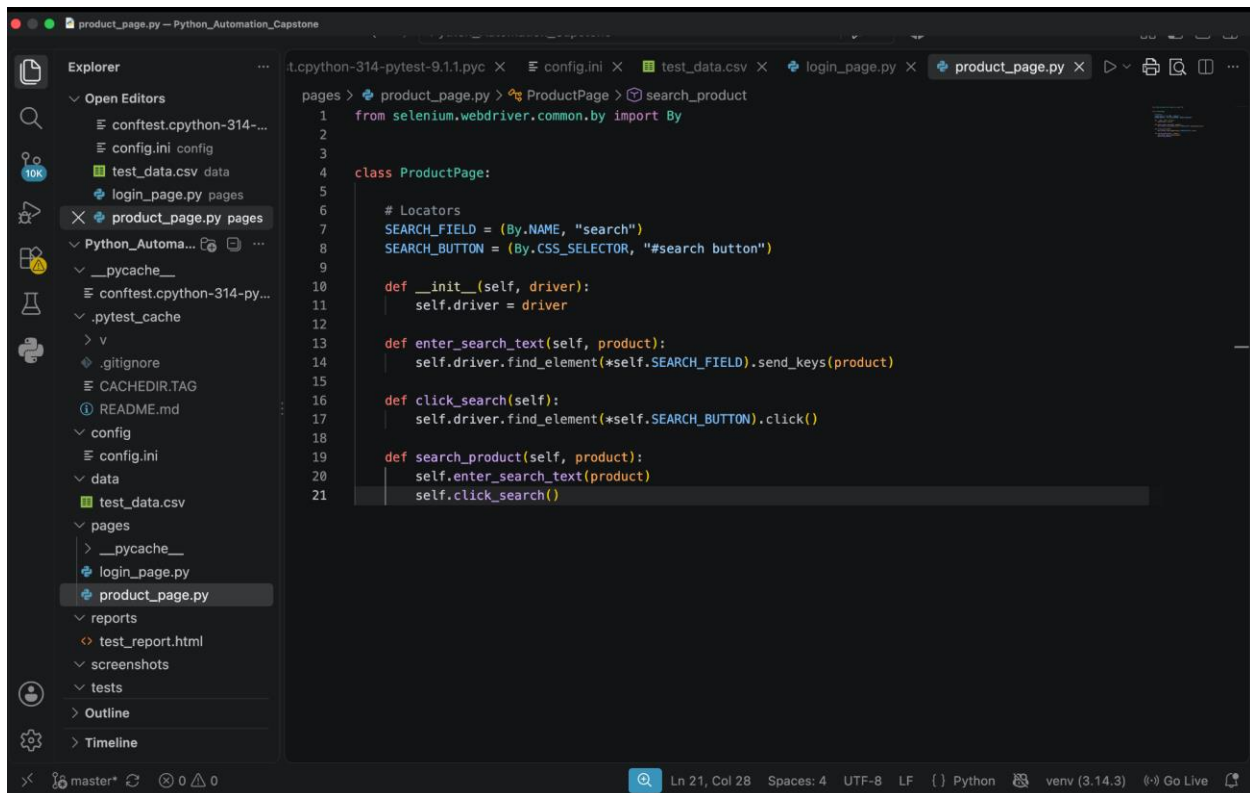
def is_logged_in(self):
    try:
        WebDriverWait(self.driver, 10).until(
            EC.url_contains("route=account/account")
        )
        return True
    except:
        return False

```





File : - product_page.py



```
from selenium.webdriver.common.by import By

class ProductPage:

    # Locators
    SEARCH_FIELD = (By.NAME, "search")
    SEARCH_BUTTON = (By.CSS_SELECTOR, "#search button")

    def __init__(self, driver):
        self.driver = driver

    def enter_search_text(self, product):
        self.driver.find_element(*self.SEARCH_FIELD).send_keys(product)

    def click_search(self):
        self.driver.find_element(*self.SEARCH_BUTTON).click()

    def search_product(self, product):
        self.enter_search_text(product)
        self.click_search()
```

Folder : - reports

File:- test_report.html

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8"/>
    <title id="head-title">test_report.html</title>
    <style type="text/css">body {
font-family: Helvetica, Arial, sans-serif;
font-size: 12px;
/* do not increase min-width as some may use split screens */
min-width: 800px;
color: #999;
}

h1 {
  font-size: 24px;
  color: black;
}

h2 {
  font-size: 16px;
  color: black;
}

p {
  color: black;
}

a {
  color: #999;
}

table {
  border-collapse: collapse;
}

/*****
* SUMMARY INFORMATION
*****/
```

```

#environment td {
  padding: 5px;
  border: 1px solid #e6e6e6;
  vertical-align: top;
}
#environment tr:nth-child(odd) {
  background-color: #f6f6f6;
}
#environment ul {
  margin: 0;
  padding: 0 20px;
}

/*****
* TEST RESULT COLORS
*****/
span.passed,
.passed .col-result {
  color: green;
}

span.skipped,
span.xfailed,
span.rerun,
.skipped .col-result,
.xfailed .col-result,
.rerun .col-result {
  color: orange;
}

span.error,
span.failed,
span.xpassed,
.error .col-result,
.failed .col-result,
.xpassed .col-result {
  color: red;
}

.col-links__extra {
  margin-right: 3px;
}

```

```

/*****
* RESULTS TABLE
*
* 1. Table Layout
* 2. Extra
* 3. Sorting items
*
*****/
/*-----
* 1. Table Layout
*-----*/
#results-table {
  border: 1px solid #e6e6e6;
  color: #999;
  font-size: 12px;
  width: 100%;
}
#results-table th,
#results-table td {
  padding: 5px;
  border: 1px solid #e6e6e6;
  text-align: left;
}
#results-table th {
  font-weight: bold;
}

/*-----
* 2. Extra
*-----*/
.logwrapper {
  max-height: 230px;
  overflow-y: scroll;
  background-color: #e6e6e6;
}
.logwrapper.expanded {
  max-height: none;
}
.logwrapper.expanded .logexpander:after {
  content: "collapse [-]";
}

```

```

.logwrapper .logexpander {
  z-index: 1;
  position: sticky;
  top: 10px;
  width: max-content;
  border: 1px solid;
  border-radius: 3px;
  padding: 5px 7px;
  margin: 10px 0 10px calc(100% - 80px);
  cursor: pointer;
  background-color: #e6e6e6;
}
.logwrapper .logexpander:after {
  content: "expand [+]";
}
.logwrapper .logexpander:hover {
  color: #000;
  border-color: #000;
}
.logwrapper .log {
  min-height: 40px;
  position: relative;
  top: -50px;
  height: calc(100% + 50px);
  border: 1px solid #e6e6e6;
  color: black;
  display: block;
  font-family: "Courier New", Courier, monospace;
  padding: 5px;
  padding-right: 80px;
  white-space: pre-wrap;
}

div.media {
  border: 1px solid #e6e6e6;
  float: right;
  height: 240px;
  margin: 0 5px;
  overflow: hidden;
  width: 320px;
}

```

```

.media-container {
  display: grid;
  grid-template-columns: 25px auto 25px;
  align-items: center;
  flex: 1 1;
  overflow: hidden;
  height: 200px;
}

.media-container--fullscreen {
  grid-template-columns: 0px auto 0px;
}

.media-container__nav--right,
.media-container__nav--left {
  text-align: center;
  cursor: pointer;
}

.media-container__viewport {
  cursor: pointer;
  text-align: center;
  height: inherit;
}

.media-container__viewport img,
.media-container__viewport video {
  object-fit: cover;
  width: 100%;
  max-height: 100%;
}

.media__name,
.media__counter {
  display: flex;
  flex-direction: row;
  justify-content: space-around;
  flex: 0 0 25px;
  align-items: center;
}

.collapsible td:not(.col-links) {
  cursor: pointer;
}

```

```

}

.collapsible td:not(.col-links):hover::after {
  color: #bbb;
  font-style: italic;
  cursor: pointer;
}

.col-result {
  width: 130px;
}

.col-result:hover::after {
  content: " (hide details)";
}

.col-result.collapsed:hover::after {
  content: " (show details)";
}

#environment-header h2:hover::after {
  content: " (hide details)";
  color: #bbb;
  font-style: italic;
  cursor: pointer;
  font-size: 12px;
}

#environment-header.collapsed h2:hover::after {
  content: " (show details)";
  color: #bbb;
  font-style: italic;
  cursor: pointer;
  font-size: 12px;
}

/*-----
* 3. Sorting items
*-----*/

.sortable {
  cursor: pointer;
}

.sortable.desc:after {
  content: " ";
}

```

```

position: relative;
left: 5px;
bottom: -12.5px;
border: 10px solid #4caf50;
border-bottom: 0;
border-left-color: transparent;
border-right-color: transparent;
}

.sortable.asc:after {
content: " ";
position: relative;
left: 5px;
bottom: 12.5px;
border: 10px solid #4caf50;
border-top: 0;
border-left-color: transparent;
border-right-color: transparent;
}

.hidden, .summary__reload__button.hidden {
display: none;
}

.summary__data {
flex: 0 0 550px;
}

.summary__reload {
flex: 1 1;
display: flex;
justify-content: center;
}

.summary__reload__button {
flex: 0 0 300px;
display: flex;
color: white;
font-weight: bold;
background-color: #4caf50;
text-align: center;
justify-content: center;
align-items: center;
border-radius: 3px;
cursor: pointer;
}

```

```

}

.summary__reload_button:hover {
  background-color: #46a049;
}

.summary__spacer {
  flex: 0 0 550px;
}

.controls {
  display: flex;
  justify-content: space-between;
}

.filters,
.collapse {
  display: flex;
  align-items: center;
}

.filters button,
.collapse button {
  color: #999;
  border: none;
  background: none;
  cursor: pointer;
  text-decoration: underline;
}

.filters button:hover,
.collapse button:hover {
  color: #ccc;
}

.filter__label {
  margin-right: 10px;
}

</style>

</head>
<body>
  <h1 id="title">test_report.html</h1>
  <p>Report generated on 15-Sep-2026 at 10:13:01 by <a
href="https://pypi.python.org/pypi/pytest-html">pytest-html</a>

```

```

v4.2.0</p>
<div id="environment-header">
  <h2>Environment</h2>
</div>
<table id="environment"></table>
<!-- TEMPLATES -->
  <template id="template_environment_row">
    <tr>
      <td></td>
      <td></td>
    </tr>
  </template>
  <template id="template_results-table__body--empty">
    <tbody class="results-table-row">
      <tr id="not-found-message">
        <td colspan="4">No results found. Check the filters.</td>
      </tr>
    </tbody>
  </template>
  <template id="template_results-table__tbody">
    <tbody class="results-table-row">
      <tr class="collapsible">
      </tr>
      <tr class="extras-row">
        <td class="extra" colspan="4">
          <div class="extraHTML"></div>
          <div class="media">
            <div class="media-container">
              <div class="media-container__nav--left">&lt;</div>
              <div class="media-container__viewport">
                <img src="" />
                <video controls>
                  <source src="" type="video/mp4">
                </video>
              </div>
              <div class="media-container__nav--right">&gt;</div>
            </div>
            <div class="media__name"></div>
            <div class="media__counter"></div>
          </div>
          <div class="logwrapper">
            <div class="logexpander"></div>

```

```

        <div class="log"></div>
    </div>
</td>
</tr>
</tbody>
</template>
<!-- END TEMPLATES -->
<div class="summary">
    <div class="summary__data">
        <h2>Summary</h2>
        <div class="additional-summary prefix">
        </div>
        <p class="run-count">4 tests took 00:00:12.</p>
        <p class="filter">(Un)check the boxes to filter the results.</p>
        <div class="summary__reload">
            <div class="summary__reload__button hidden" onclick="location.reload()">
                <div>There are still tests running. <br />Reload this page to get the
latest results!</div>
            </div>
        </div>
        <div class="summary__spacer"></div>
        <div class="controls">
            <div class="filters">
                <input checked="true" class="filter" name="filter_checkbox" type="checkbox"
data-test-result="failed" disabled>
                <span class="failed">0 Failed,</span>
                <input checked="true" class="filter" name="filter_checkbox" type="checkbox"
data-test-result="passed" >
                <span class="passed">4 Passed,</span>
                <input checked="true" class="filter" name="filter_checkbox" type="checkbox"
data-test-result="skipped" disabled>
                <span class="skipped">0 Skipped,</span>
                <input checked="true" class="filter" name="filter_checkbox" type="checkbox"
data-test-result="xfailed" disabled>
                <span class="xfailed">0 Expected failures,</span>
                <input checked="true" class="filter" name="filter_checkbox" type="checkbox"
data-test-result="xpassed" disabled>
                <span class="xpassed">0 Unexpected passes,</span>
                <input checked="true" class="filter" name="filter_checkbox" type="checkbox"
data-test-result="error" disabled>
                <span class="error">0 Errors,</span>
            </div>
        </div>
    </div>

```

```

        <input checked="true" class="filter" name="filter_checkbox" type="checkbox"
data-test-result="rerun" disabled>
        <span class="rerun">0 Reruns</span>
        <input checked="true" class="filter" name="filter_checkbox" type="checkbox"
data-test-result="retried" disabled>
        <span class="retried">0 Retried,</span>
    </div>
    <div class="collapse">
        <button id="show_all_details">Show all details</button>&nbsp;&nbsp;<button
id="hide_all_details">Hide all details</button>
    </div>
</div>
<div class="additional-summary summary">
</div>
<div class="additional-summary postfix">
</div>
</div>
<table id="results-table">
    <thead id="results-table-head">
        <tr>
            <th class="sortable" data-column-type="result">Result</th>
            <th class="sortable" data-column-type="testId">Test</th>
            <th class="sortable" data-column-type="duration">Duration</th>
            <th>Links</th>
        </tr>
    </thead>
</table>
<footer>
    <div id="data-container" data-jsonblob="{&#34;environment&#34;: {&#34;Python&#34;:
&#34;3.14.3&#34;, &#34;Platform&#34;: &#34;macOS-26.6.2-arm64-arm-64bit-Mach-O&#34;,
&#34;Packages&#34;: {&#34;pytest&#34;: &#34;9.1.1&#34;, &#34;pluggy&#34;:
&#34;1.6.0&#34;}, &#34;Plugins&#34;: {&#34;metadata&#34;: &#34;3.1.1&#34;,
&#34;html&#34;: &#34;4.2.0&#34;}, &#34;JAVA_HOME&#34;:
&#34;/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home&#34;},
&#34;tests&#34;: {&#34;tests/test_login.py::test_valid_login&#34;: [{&#34;extras&#34;:
[], &#34;result&#34;: &#34;Passed&#34;, &#34;testId&#34;:
&#34;tests/test_login.py::test_valid_login&#34;, &#34;duration&#34;:
&#34;00:00:04&#34;, &#34;resultsTableRow&#34;: [&#34;&lt;td class=\&#34;col-
result\&#34;&gt;Passed&lt;/td&gt;&#34;, &#34;&lt;td class=\&#34;col-
testId\&#34;&gt;tests/test_login.py::test_valid_login&lt;/td&gt;&#34;, &#34;&lt;td
class=\&#34;col-duration\&#34;&gt;00:00:04&lt;/td&gt;&#34;, &#34;&lt;td

```

```

class=\&#34;col-links\&#34;&gt;&lt;/td>&#34;], &#34;log&#34;: &#34;No log output
captured.&#34;]], &#34;tests/test_login_unittest.py::TestLogin::test_valid_login&#34;:
[{\&#34;extras&#34;: [], &#34;result&#34;: &#34;Passed&#34;, &#34;testId&#34;:
&#34;tests/test_login_unittest.py::TestLogin::test_valid_login&#34;,
&#34;duration&#34;: &#34;00:00:03&#34;, &#34;resultsTableRow&#34;: [{\&#34;&lt;td
class=\&#34;col-result\&#34;&gt;Passed&lt;/td>&#34;, &#34;&lt;td class=\&#34;col-
testId\&#34;&gt;tests/test_login_unittest.py::TestLogin::test_valid_login&lt;/td>&#34;,
&#34;&lt;td class=\&#34;col-duration\&#34;&gt;00:00:03&lt;/td>&#34;,
&#34;&lt;td class=\&#34;col-links\&#34;&gt;&lt;/td>&#34;], &#34;log&#34;: &#34;No
log output captured.&#34;]],
&#34;tests/test_product_search.py::test_product_search&#34;: [{\&#34;extras&#34;: [],
&#34;result&#34;: &#34;Passed&#34;, &#34;testId&#34;:
&#34;tests/test_product_search.py::test_product_search&#34;, &#34;duration&#34;:
&#34;00:00:03&#34;, &#34;resultsTableRow&#34;: [{\&#34;&lt;td class=\&#34;col-
result\&#34;&gt;Passed&lt;/td>&#34;, &#34;&lt;td class=\&#34;col-
testId\&#34;&gt;tests/test_product_search.py::test_product_search&lt;/td>&#34;,
&#34;&lt;td class=\&#34;col-duration\&#34;&gt;00:00:03&lt;/td>&#34;, &#34;&lt;td
class=\&#34;col-links\&#34;&gt;&lt;/td>&#34;], &#34;log&#34;: &#34;No log output
captured.&#34;]], &#34;tests/test_setup.py::test_open_application&#34;:
[{\&#34;extras&#34;: [], &#34;result&#34;: &#34;Passed&#34;, &#34;testId&#34;:
&#34;tests/test_setup.py::test_open_application&#34;, &#34;duration&#34;:
&#34;00:00:02&#34;, &#34;resultsTableRow&#34;: [{\&#34;&lt;td class=\&#34;col-
result\&#34;&gt;Passed&lt;/td>&#34;, &#34;&lt;td class=\&#34;col-
testId\&#34;&gt;tests/test_setup.py::test_open_application&lt;/td>&#34;,
&#34;&lt;td class=\&#34;col-duration\&#34;&gt;00:00:02&lt;/td>&#34;, &#34;&lt;td
class=\&#34;col-links\&#34;&gt;&lt;/td>&#34;], &#34;log&#34;: &#34;No log output
captured.&#34;]], &#34;renderCollapsed&#34;: [{\&#34;passed&#34;},
&#34;initialSort&#34;: &#34;result&#34;, &#34;title&#34;:
&#34;test_report.html&#34;}&#34;]></div>
<script>
    (function(){function r(e,n,t){function o(i,f){if(!n[i]){if(!e[i]){var
c="function"==typeof require&&require;if(!f&&c)return c(i,!0);if(u)return u(i,!0);var
a=new Error("Cannot find module '"+i+"'");throw a.code="MODULE_NOT_FOUND",a}var
p=n[i]={exports:{}};e[i][0].call(p.exports,function(r){var n=e[i][1][r];return
o(n||r)},p,p.exports,r,e,n,t)}return n[i].exports}for(var u="function"==typeof
require&&require,i=0;i<t.length;i++)o(t[i]);return o}r})()({1:[function(require,module,exports){
const { getCollapsedCategory, setCollapsedIds } = require('./storage.js')

class DataManager {
    setManager(data) {
        const collapsedCategories = [...getCollapsedCategory(data.renderCollapsed)]

```

```

const collapsedIds = []
const tests = Object.values(data.tests).flat().map((test, index) => {
  const collapsed = collapsedCategories.includes(test.result.toLowerCase())
  const id = `test_${index}`
  if (collapsed) {
    collapsedIds.push(id)
  }
  return {
    ...test,
    id,
    collapsed,
  }
})
const dataBlob = { ...data, tests }
this.data = { ...dataBlob }
this.renderData = { ...dataBlob }
setCollapsedIds(collapsedIds)
}

get allData() {
  return { ...this.data }
}

resetRender() {
  this.renderData = { ...this.data }
}

setRender(data) {
  this.renderData.tests = [...data]
}

toggleCollapsedItem(id) {
  this.renderData.tests = this.renderData.tests.map((test) =>
    test.id === id ? { ...test, collapsed: !test.collapsed } : test,
  )
}

set allCollapsed(collapsed) {
  this.renderData = { ...this.renderData, tests:
[...this.renderData.tests.map((test) => (
    { ...test, collapsed }
  ))] }

```

```

    }

    get testSubset() {
        return [...this.renderData.tests]
    }

    get environment() {
        return this.renderData.environment
    }

    get initialSort() {
        return this.data.initialSort
    }
}

module.exports = {
    manager: new DataManager(),
}

}, {"/storage.js":8}], 2: [function(require, module, exports) {
const mediaViewer = require('./mediaviewer.js')
const templateEnvRow = document.getElementById('template_environment_row')
const templateResult = document.getElementById('template_results-table__tbody')

function htmlToElements(html) {
    const temp = document.createElement('template')
    temp.innerHTML = html
    return temp.content.childNodes
}

const find = (selector, elem) => {
    if (!elem) {
        elem = document
    }
    return elem.querySelector(selector)
}

const findAll = (selector, elem) => {
    if (!elem) {
        elem = document
    }
    return [...elem.querySelectorAll(selector)]
}

```

```

}

const dom = {
  getStaticRow: (key, value) => {
    const envRow = templateEnvRow.content.cloneNode(true)
    const isObj = typeof value === 'object' && value !== null
    const values = isObj ? Object.keys(value).map((k) => `${k}: ${value[k]}`) :
null

    const valuesElement = htmlToElements(
      values ? `<ul>${values.map((val) => `<li>${val}</li>`).join('')}</ul>` :
`<div>${value}</div>`)[0]
    const td = findAll('td', envRow)
    td[0].textContent = key
    td[1].appendChild(valuesElement)

    return envRow
  },
  getResultTBody: ({ testId, id, log, extras, resultsTableRow, tableHtml, result,
collapsed }) => {
    const resultBody = templateResult.content.cloneNode(true)
    resultBody.querySelector('tbody').classList.add(result.toLowerCase())
    resultBody.querySelector('tbody').id = testId
    resultBody.querySelector('.collapsible').dataset.id = id

    resultsTableRow.forEach((html) => {
      const t = document.createElement('template')
      t.innerHTML = html
      resultBody.querySelector('.collapsible').appendChild(t.content)
    })

    if (log) {
      // Wrap lines starting with "E" with span.error to color those lines red
      const wrappedLog = log.replace(/^E.*$/gm, (match) => `<span
class="error">${match}</span>`)
      resultBody.querySelector('.log').innerHTML = wrappedLog
    } else {
      resultBody.querySelector('.log').remove()
    }

    if (collapsed) {

```

```

        resultBody.querySelector('.collapsible > .col-
result')?.classList.add('collapsed')
        resultBody.querySelector('.extras-row').classList.add('hidden')
    } else {
        resultBody.querySelector('.collapsible > .col-
result')?.classList.remove('collapsed')
    }

    const media = []
    extras?.forEach(({ name, format_type, content }) => {
        if (['image', 'video'].includes(format_type)) {
            media.push({ path: content, name, format_type })
        }

        if (format_type === 'html') {
            resultBody.querySelector('.extraHTML').insertAdjacentHTML('beforeend',
`<div>${content}</div>`)
        }
    })
    mediaViewer.setup(resultBody, media)

    // Add custom html from the pytest_html_results_table_html hook
    tableHtml?.forEach((item) => {

resultBody.querySelector('td[class="extra"]').insertAdjacentHTML('beforeend', item)
    })

    return resultBody
},
}

module.exports = {
    dom,
    htmlToElements,
    find,
    findAll,
}

}, {"/mediaviewer.js":6}], 3:[function(require,module,exports){
const { manager } = require('./datamanager.js')
const { doSort } = require('./sort.js')
const storageModule = require('./storage.js')

```

```

const getFilteredSubSet = (filter) =>
  manager.allData.tests.filter(({ result }) => filter.includes(result.toLowerCase()))

const doInitFilter = () => {
  const currentFilter = storageModule.getVisible()
  const filteredSubset = getFilteredSubSet(currentFilter)
  manager.setRender(filteredSubset)
}

const doFilter = (type, show) => {
  if (show) {
    storageModule.showCategory(type)
  } else {
    storageModule.hideCategory(type)
  }

  const currentFilter = storageModule.getVisible()
  const filteredSubset = getFilteredSubSet(currentFilter)
  manager.setRender(filteredSubset)

  const sortColumn = storageModule.getSort()
  doSort(sortColumn, true)
}

module.exports = {
  doFilter,
  doInitFilter,
}

}, {"/datamanager.js":1,"./sort.js":7,"./storage.js":8}],4:[function(require,module,exports){
const { redraw, bindEvents, renderStatic } = require('./main.js')
const { doInitFilter } = require('./filter.js')
const { doInitSort } = require('./sort.js')
const { manager } = require('./datamanager.js')
const data = JSON.parse(document.getElementById('data-container').dataset.jsonblob)

function init() {
  manager.setManager(data)
  doInitFilter()
  doInitSort()
}

```

```

    renderStatic()
    redraw()
    bindEvents()
}

init()

}, { './datamanager.js': 1, './filter.js': 3, './main.js': 5, './sort.js': 7 }], 5: [function(require, module, exports) {
const { dom, find, findAll } = require('./dom.js')
const { manager } = require('./datamanager.js')
const { doSort } = require('./sort.js')
const { doFilter } = require('./filter.js')
const {
    getVisible,
    getCollapsedIds,
    setCollapsedIds,
    getSort,
    getSortDirection,
    possibleFilters,
} = require('./storage.js')

const removeChildren = (node) => {
    while (node.firstChild) {
        node.removeChild(node.firstChild)
    }
}

const renderStatic = () => {
    const renderEnvironmentTable = () => {
        const environment = manager.environment
        const rows = Object.keys(environment).map((key) => dom.getStaticRow(key, environment[key]))
        const table = document.getElementById('environment')
        removeChildren(table)
        rows.forEach((row) => table.appendChild(row))
    }
    renderEnvironmentTable()
}

const addItemToggleListener = (elem) => {
    elem.addEventListener('click', ({ target }) => {

```

```

    const id = target.parentElement.dataset.id
    manager.toggleCollapsedItem(id)

    const collapsedIds = getCollapsedIds()
    if (collapsedIds.includes(id)) {
        const updated = collapsedIds.filter((item) => item !== id)
        setCollapsedIds(updated)
    } else {
        collapsedIds.push(id)
        setCollapsedIds(collapsedIds)
    }
    redraw()
})
}

const renderContent = (tests) => {
    const sortAttr = getSort(manager.initialSort)
    const sortAsc = JSON.parse(getSortDirection())
    const rows = tests.map(dom.getResultTBody)
    const table = document.getElementById('results-table')
    const tableHeader = document.getElementById('results-table-head')

    const newTable = document.createElement('table')
    newTable.id = 'results-table'

    // remove all sorting classes and set the relevant
    findAll('.sortable', tableHeader).forEach((elem) => elem.classList.remove('asc',
'desc'))
    tableHeader.querySelector(`.sortable[data-column-
type="${sortAttr}"]`)?.classList.add(sortAsc ? 'desc' : 'asc')
    newTable.appendChild(tableHeader)

    if (!rows.length) {
        const emptyTable = document.getElementById('template_results-table__body--
empty').content.cloneNode(true)
        newTable.appendChild(emptyTable)
    } else {
        rows.forEach((row) => {
            if (!!row) {
                findAll('.collapsible td:not(.col-links',
row).forEach(addItemToggleListener)
                find('.logexpander', row).addEventListener('click',

```

```

        (evt) => evt.target.parentNode.classList.toggle('expanded'),
    )
    newTable.appendChild(row)
  }
})
}

table.replaceWith(newTable)
}

const renderDerived = () => {
  const currentFilter = getVisible()
  possibleFilters.forEach((result) => {
    const input = document.querySelector(`input[data-test-result="${result}"]`)
    input.checked = currentFilter.includes(result)
  })
}

const bindEvents = () => {
  const filterColumn = (evt) => {
    const { target: element } = evt
    const { testResult } = element.dataset

    doFilter(testResult, element.checked)
    const collapsedIds = getCollapsedIds()
    const updated = manager.renderData.tests.map((test) => {
      return {
        ...test,
        collapsed: collapsedIds.includes(test.id),
      }
    })
    manager.setRender(updated)
    redraw()
  }

  const header = document.getElementById('environment-header')
  header.addEventListener('click', () => {
    const table = document.getElementById('environment')
    table.classList.toggle('hidden')
    header.classList.toggle('collapsed')
  })
}

```

```

    findAll('input[name="filter_checkbox"]').forEach((elem) => {
        elem.addEventListener('click', filterColumn)
    })

    findAll('.sortable').forEach((elem) => {
        elem.addEventListener('click', (evt) => {
            const { target: element } = evt
            const { columnType } = element.dataset
            doSort(columnType)
            redraw()
        })
    })

    document.getElementById('show_all_details').addEventListener('click', () => {
        manager.allCollapsed = false
        setCollapsedIds([])
        redraw()
    })

    document.getElementById('hide_all_details').addEventListener('click', () => {
        manager.allCollapsed = true
        const allIds = manager.renderData.tests.map((test) => test.id)
        setCollapsedIds(allIds)
        redraw()
    })
}

const redraw = () => {
    const { testSubset } = manager

    renderContent(testSubset)
    renderDerived()
}

module.exports = {
    redraw,
    bindEvents,
    renderStatic,
}

}, {"/datamanager.js":1,"./dom.js":2,"./filter.js":3,"./sort.js":7,"./storage.js":8}],
6:[function(require,module,exports){
class MediaViewer {

```

```

    constructor(assets) {
        this.assets = assets
        this.index = 0
    }

    nextActive() {
        this.index = this.index === this.assets.length - 1 ? 0 : this.index + 1
        return [this.activeFile, this.index]
    }

    prevActive() {
        this.index = this.index === 0 ? this.assets.length - 1 : this.index - 1
        return [this.activeFile, this.index]
    }

    get currentIndex() {
        return this.index
    }

    get activeFile() {
        return this.assets[this.index]
    }
}

const setup = (resultBody, assets) => {
    if (!assets.length) {
        resultBody.querySelector('.media').classList.add('hidden')
        return
    }

    const mediaViewer = new MediaViewer(assets)
    const container = resultBody.querySelector('.media-container')
    const leftArrow = resultBody.querySelector('.media-container__nav--left')
    const rightArrow = resultBody.querySelector('.media-container__nav--right')
    const mediaName = resultBody.querySelector('.media__name')
    const counter = resultBody.querySelector('.media__counter')
    const imageEl = resultBody.querySelector('img')
    const sourceEl = resultBody.querySelector('source')
    const videoEl = resultBody.querySelector('video')

    const setImg = (media, index) => {

```

```

    if (media?.format_type === 'image') {
      imageEl.src = media.path

      imageEl.classList.remove('hidden')
      videoEl.classList.add('hidden')
    } else if (media?.format_type === 'video') {
      sourceEl.src = media.path

      videoEl.classList.remove('hidden')
      imageEl.classList.add('hidden')
    }

    mediaName.innerText = media?.name
    counter.innerText = `${index + 1} / ${assets.length}`
  }
  setImg(mediaViewer.activeFile, mediaViewer.currentIndex)

  const moveLeft = () => {
    const [media, index] = mediaViewer.prevActive()
    setImg(media, index)
  }
  const doRight = () => {
    const [media, index] = mediaViewer.nextActive()
    setImg(media, index)
  }
  const openImg = () => {
    window.open(mediaViewer.activeFile.path, '_blank')
  }
  if (assets.length === 1) {
    container.classList.add('media-container--fullscreen')
  } else {
    leftArrow.addEventListener('click', moveLeft)
    rightArrow.addEventListener('click', doRight)
  }
  imageEl.addEventListener('click', openImg)
}

module.exports = {
  setup,
}

}, {}, 7: [function (require, module, exports) {

```

```

const { manager } = require('./datamanager.js')
const storageModule = require('./storage.js')

const genericSort = (list, key, ascending, customOrder) => {
  let sorted
  if (customOrder) {
    sorted = list.sort((a, b) => {
      const aValue = a.result.toLowerCase()
      const bValue = b.result.toLowerCase()

      const aIndex = customOrder.findIndex((item) => item.toLowerCase() ===
aValue)
      const bIndex = customOrder.findIndex((item) => item.toLowerCase() ===
bValue)

      // Compare the indices to determine the sort order
      return aIndex - bIndex
    })
  } else {
    sorted = list.sort((a, b) => a[key] === b[key] ? 0 : a[key] > b[key] ? 1 : -1)
  }

  if (ascending) {
    sorted.reverse()
  }
  return sorted
}

const durationSort = (list, ascending) => {
  const parseDuration = (duration) => {
    if (duration.includes(':')) {
      // If it's in the format "HH:mm:ss"
      const [hours, minutes, seconds] = duration.split(':').map(Number)
      return (hours * 3600 + minutes * 60 + seconds) * 1000
    } else {
      // If it's in the format "nnn ms"
      return parseInt(duration)
    }
  }

  const sorted = list.sort((a, b) => parseDuration(a['duration']) -
parseDuration(b['duration']))
  if (ascending) {

```

```

        sorted.reverse()
    }
    return sorted
}

const doInitSort = () => {
    const type = storageModule.getSort(manager.initialSort)
    const ascending = storageModule.getSortDirection()
    const list = manager.testSubset
    const initialOrder = ['Error', 'Failed', 'Rerun', 'XFailed', 'XPassed', 'Skipped',
'Passed']

    storageModule.setSort(type)
    storageModule.setSortDirection(ascending)

    if (type?.toLowerCase() === 'original') {
        manager.setRender(list)
    } else {
        let sortedList
        switch (type) {
            case 'duration':
                sortedList = durationSort(list, ascending)
                break
            case 'result':
                sortedList = genericSort(list, type, ascending, initialOrder)
                break
            default:
                sortedList = genericSort(list, type, ascending)
                break
        }
        manager.setRender(sortedList)
    }
}

const doSort = (type, skipDirection) => {
    const newSortType = storageModule.getSort(manager.initialSort) !== type
    const currentAsc = storageModule.getSortDirection()
    let ascending
    if (skipDirection) {
        ascending = currentAsc
    } else {
        ascending = newSortType ? false : !currentAsc
    }
}

```

```

    }

    storageModule.setSort(type)
    storageModule.setSortDirection(ascending)

    const list = manager.testSubset
    const sortedList = type === 'duration' ? durationSort(list, ascending) :
genericSort(list, type, ascending)
    manager.setRender(sortedList)
}

module.exports = {
    doInitSort,
    doSort,
}

}, {"/datamanager.js":1,"./storage.js":8}],8:[function(require,module,exports){
const possibleFilters = [
    'passed',
    'skipped',
    'failed',
    'error',
    'xfailed',
    'xpassed',
    'rerun',
]

const getVisible = () => {
    const url = new URL(window.location.href)
    const settings = new URLSearchParams(url.search).get('visible')
    const lower = (item) => {
        const lowerItem = item.toLowerCase()
        if (possibleFilters.includes(lowerItem)) {
            return lowerItem
        }
        return null
    }
    return settings === null ?
        possibleFilters :
        [...new Set(settings?.split(',').map(lower).filter((item) => item))]
}

const hideCategory = (categoryToHide) => {

```

```

    const url = new URL(window.location.href)
    const visibleParams = new URLSearchParams(url.search).get('visible')
    const currentVisible = visibleParams ? visibleParams.split(',') :
[...possibleFilters]
    const settings = [...new Set(currentVisible)].filter((f) => f !==
categoryToHide).join(',')

    url.searchParams.set('visible', settings)
    window.history.pushState({}, null, unescape(url.href))
}

const showCategory = (categoryToShow) => {
  if (typeof window === 'undefined') {
    return
  }

  const url = new URL(window.location.href)
  const currentVisible = new
URLSearchParams(url.search).get('visible')?.split(',').filter(Boolean) ||
[...possibleFilters]
  const settings = [...new Set([categoryToShow, ...currentVisible])]
  const noFilter = possibleFilters.length === settings.length || !settings.length

  noFilter ? url.searchParams.delete('visible') : url.searchParams.set('visible',
settings.join(','))
  window.history.pushState({}, null, unescape(url.href))
}

const getSort = (initialSort) => {
  const url = new URL(window.location.href)
  let sort = new URLSearchParams(url.search).get('sort')
  if (!sort) {
    sort = initialSort || 'result'
  }
  return sort
}

const setSort = (type) => {
  const url = new URL(window.location.href)
  url.searchParams.set('sort', type)
  window.history.pushState({}, null, unescape(url.href))
}

```

```

const getCollapsedCategory = (renderCollapsed) => {
  let categories
  if (typeof window !== 'undefined') {
    const url = new URL(window.location.href)
    const collapsedItems = new URLSearchParams(url.search).get('collapsed')
    switch (true) {
      case !renderCollapsed && collapsedItems === null:
        categories = ['passed']
        break
      case collapsedItems?.length === 0 || /^["']{2}$/.test(collapsedItems):
        categories = []
        break
      case /^all$/.test(collapsedItems) || collapsedItems === null &&
/^all$/.test(renderCollapsed):
        categories = [...possibleFilters]
        break
      default:
        categories = collapsedItems?.split(',').map((item) => item.toLowerCase())
    }
  }
  || renderCollapsed
    break
  }
} else {
  categories = []
}
return categories
}

const getSortDirection = () => JSON.parse(sessionStorage.getItem('sortAsc')) || false
const setSortDirection = (ascending) => sessionStorage.setItem('sortAsc', ascending)

const getCollapsedIds = () => JSON.parse(sessionStorage.getItem('collapsedIds')) || []
const setCollapsedIds = (list) => sessionStorage.setItem('collapsedIds',
JSON.stringify(list))

module.exports = {
  getVisible,
  hideCategory,
  showCategory,
  getCollapsedIds,
  setCollapsedIds,
  getSort,
  setSort,

```

```

        getSortDirection,
        setSortDirection,
        getCollapsedCategory,
        possibleFilters,
    }

}, {}]], {}, [4]);

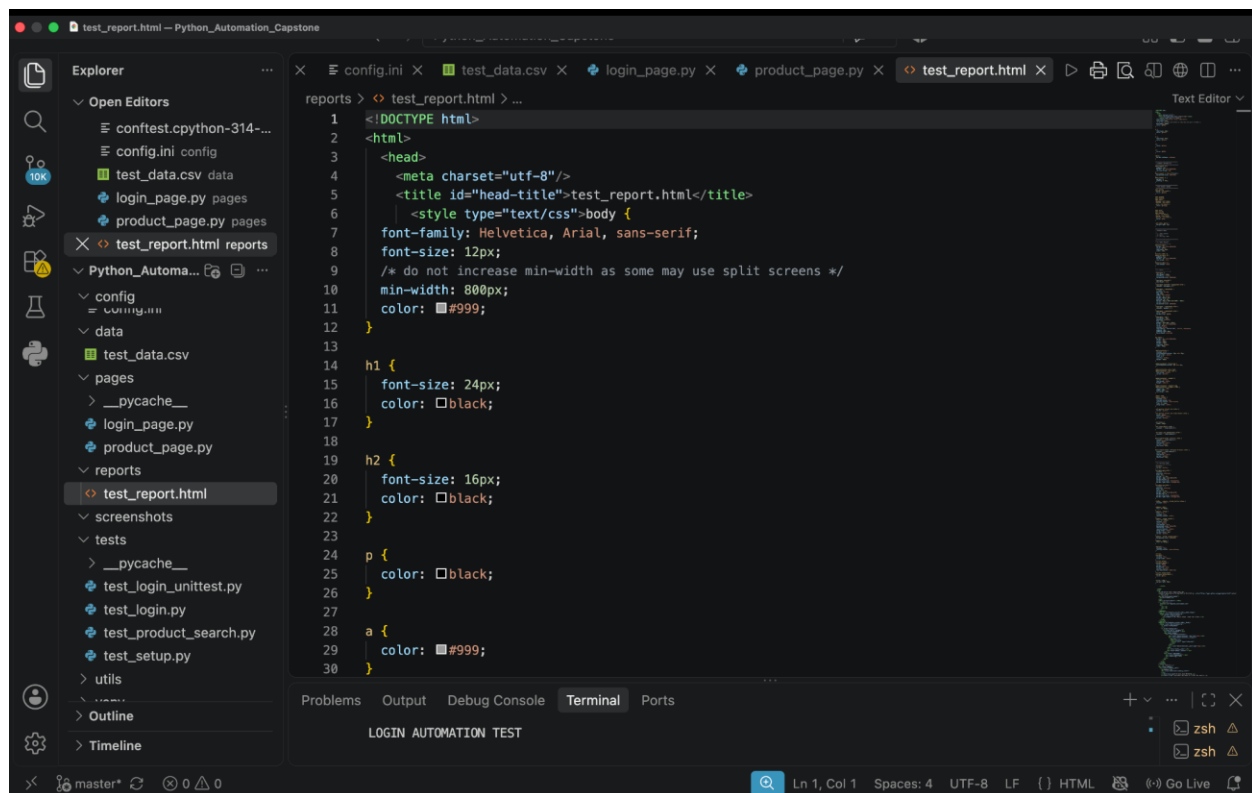
</script>

</footer>

</body>

</html>

```



Folder:- tests

File:- test_login_unittest.py

Code:-

```

import unittest

from selenium import webdriver

from pages.login_page import LoginPage

```

```

from utils.config_reader import ConfigReader
from utils.csv_reader import CSVReader

class TestLogin(unittest.TestCase):

    def setUp(self):
        print("\n=====")
        print("            UNITTEST LOGIN AUTOMATION")
        print("=====")

        print("[1] Loading configuration...")
        config = ConfigReader()
        print("[PASS] Configuration loaded")

        print("[2] Launching Chrome browser...")
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()
        self.driver.get(config.get_application_url())
        print("[PASS] Application opened")

    def test_valid_login(self):
        print("[3] Reading login data from CSV...")
        test_data = CSVReader.read_test_data()[0]
        print("[PASS] CSV data loaded")

        print("[4] Performing login...")
        login_page = LoginPage(self.driver)

        login_page.open_login_page()

        login_page.login(
            test_data["email"],
            test_data["password"]
        )

        print("[5] Verifying login...")
        self.assertTrue(login_page.is_logged_in())
        print("[PASS] Login successful")

        print("=====")
        print("            UNITTEST LOGIN TEST PASSED")

```

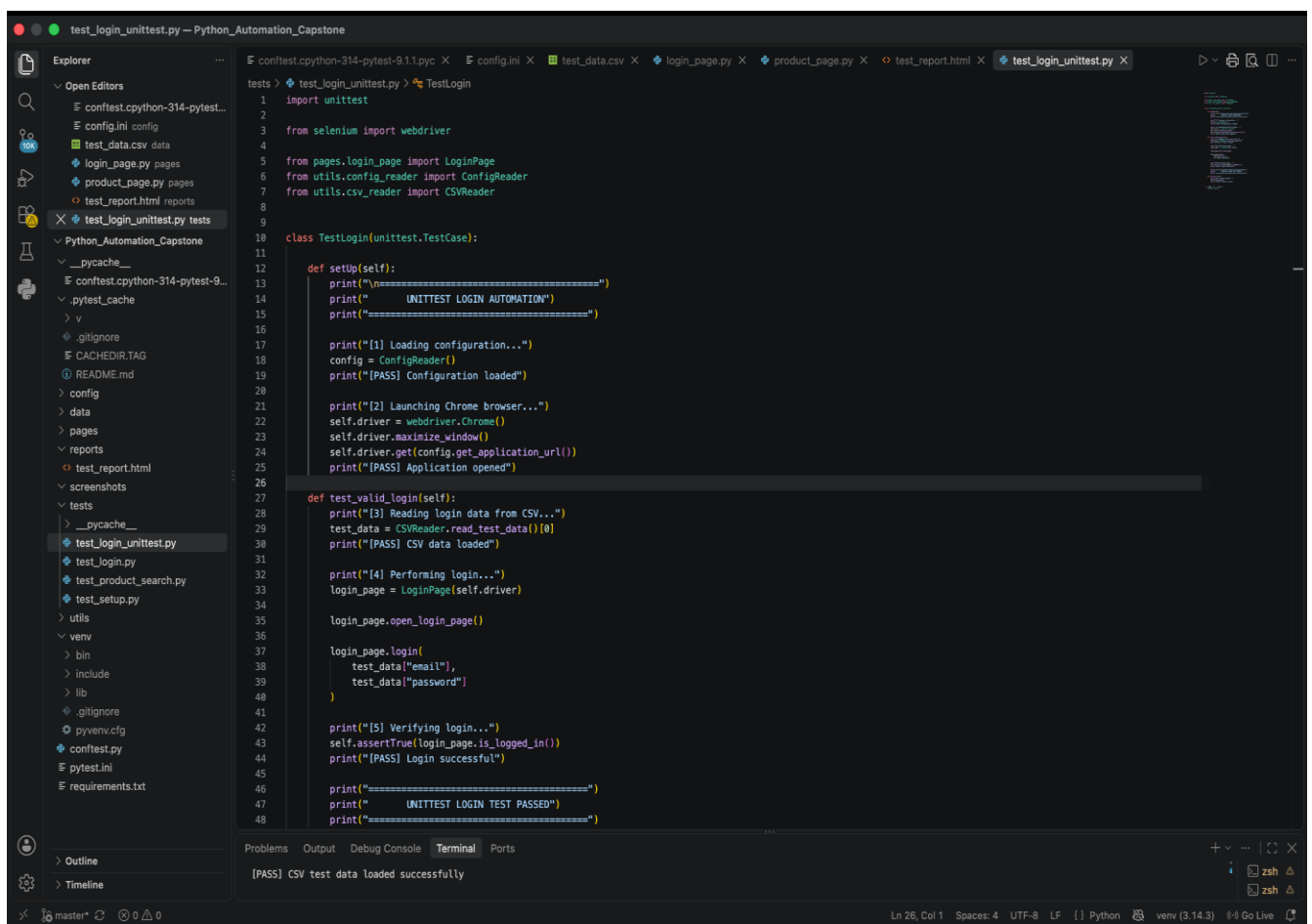
```

print("=====")

def tearDown(self):
    print("[6] Closing browser...")
    self.driver.quit()
    print("[PASS] Browser closed")

if __name__ == "__main__":
    unittest.main()

```



File:- test_login.py

```

from pages.login_page import LoginPage
from utils.csv_reader import CSVReader

```

```

def test_valid_login(driver):
    print("\n=====")
    print("          LOGIN AUTOMATION TEST")
    print("=====")

    print("[1] Reading login data from CSV...")
    test_data = CSVReader.read_test_data()[0]
    print("[PASS] CSV test data loaded successfully")

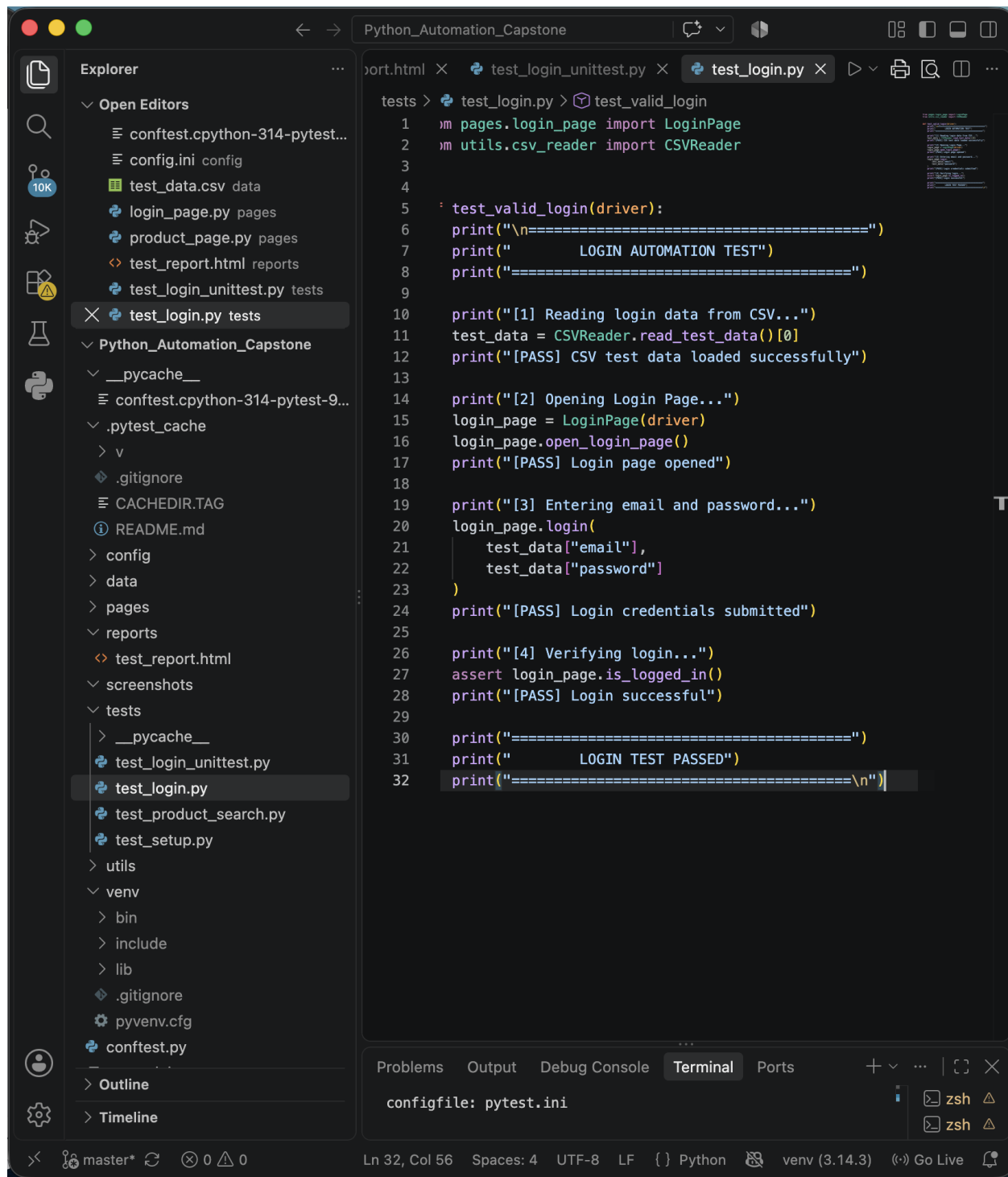
    print("[2] Opening Login Page...")
    login_page = LoginPage(driver)
    login_page.open_login_page()
    print("[PASS] Login page opened")

    print("[3] Entering email and password...")
    login_page.login(
        test_data["email"],
        test_data["password"]
    )
    print("[PASS] Login credentials submitted")

    print("[4] Verifying login...")
    assert login_page.is_logged_in()
    print("[PASS] Login successful")

    print("=====")
    print("          LOGIN TEST PASSED")
    print("=====\\n")

```



File:- test_product_search.py

```
from pages.product_page import ProductPage
from utils.csv_reader import CSVReader
```

```

def test_product_search(driver):
    print("\n=====")
    print("          PRODUCT SEARCH AUTOMATION TEST")
    print("=====")

    print("[1] Reading product data from CSV...")
    test_data = CSVReader.read_test_data()[0]
    print(f"[PASS] Product loaded: {test_data['product']}")

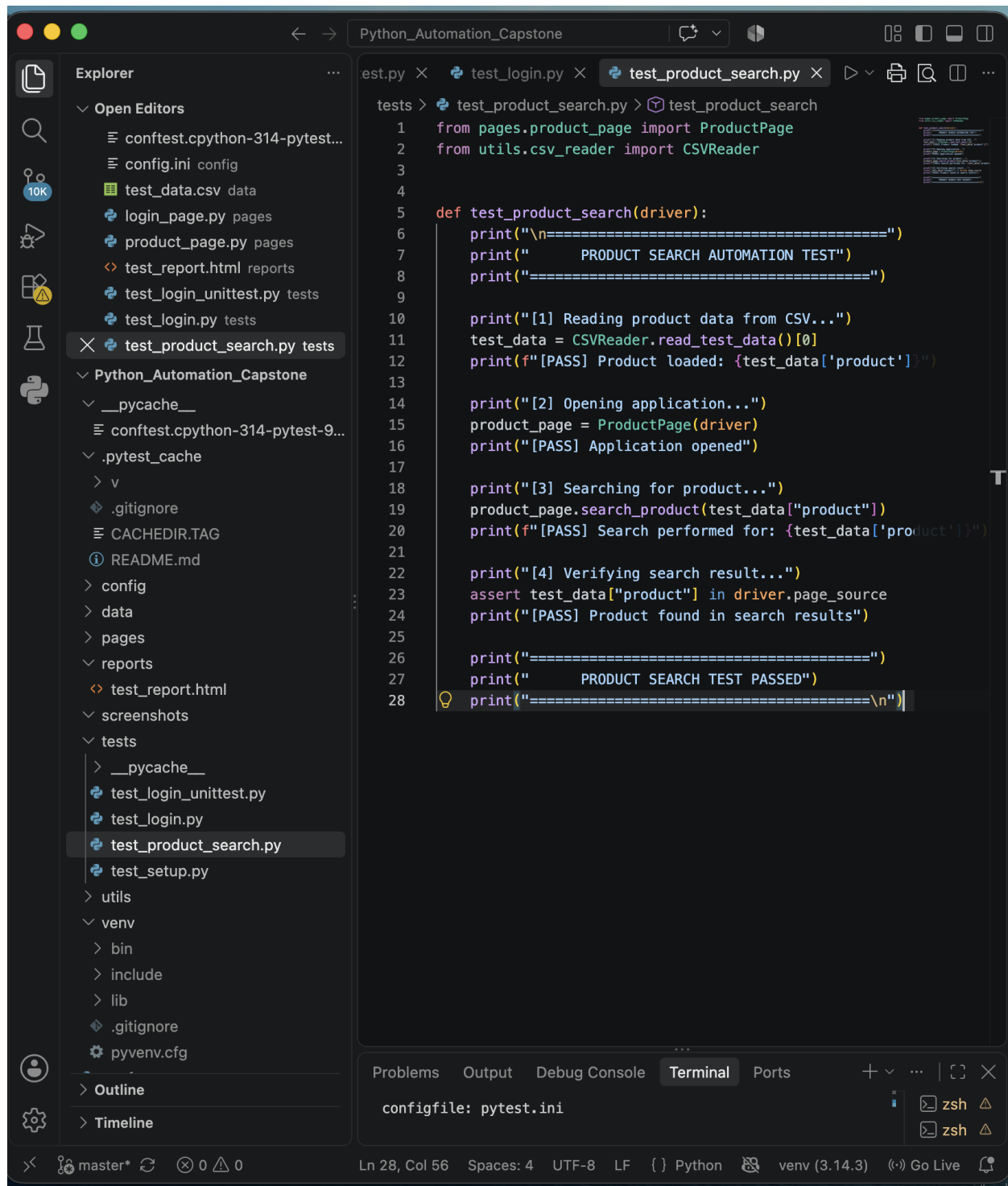
    print("[2] Opening application...")
    product_page = ProductPage(driver)
    print("[PASS] Application opened")

    print("[3] Searching for product...")
    product_page.search_product(test_data["product"])
    print(f"[PASS] Search performed for: {test_data['product']}")

    print("[4] Verifying search result...")
    assert test_data["product"] in driver.page_source
    print("[PASS] Product found in search results")

    print("=====")
    print("          PRODUCT SEARCH TEST PASSED")
    print("=====\\n")

```



File:- test_setup.py

```
def test_open_application(driver):
    print("\n=====")
```

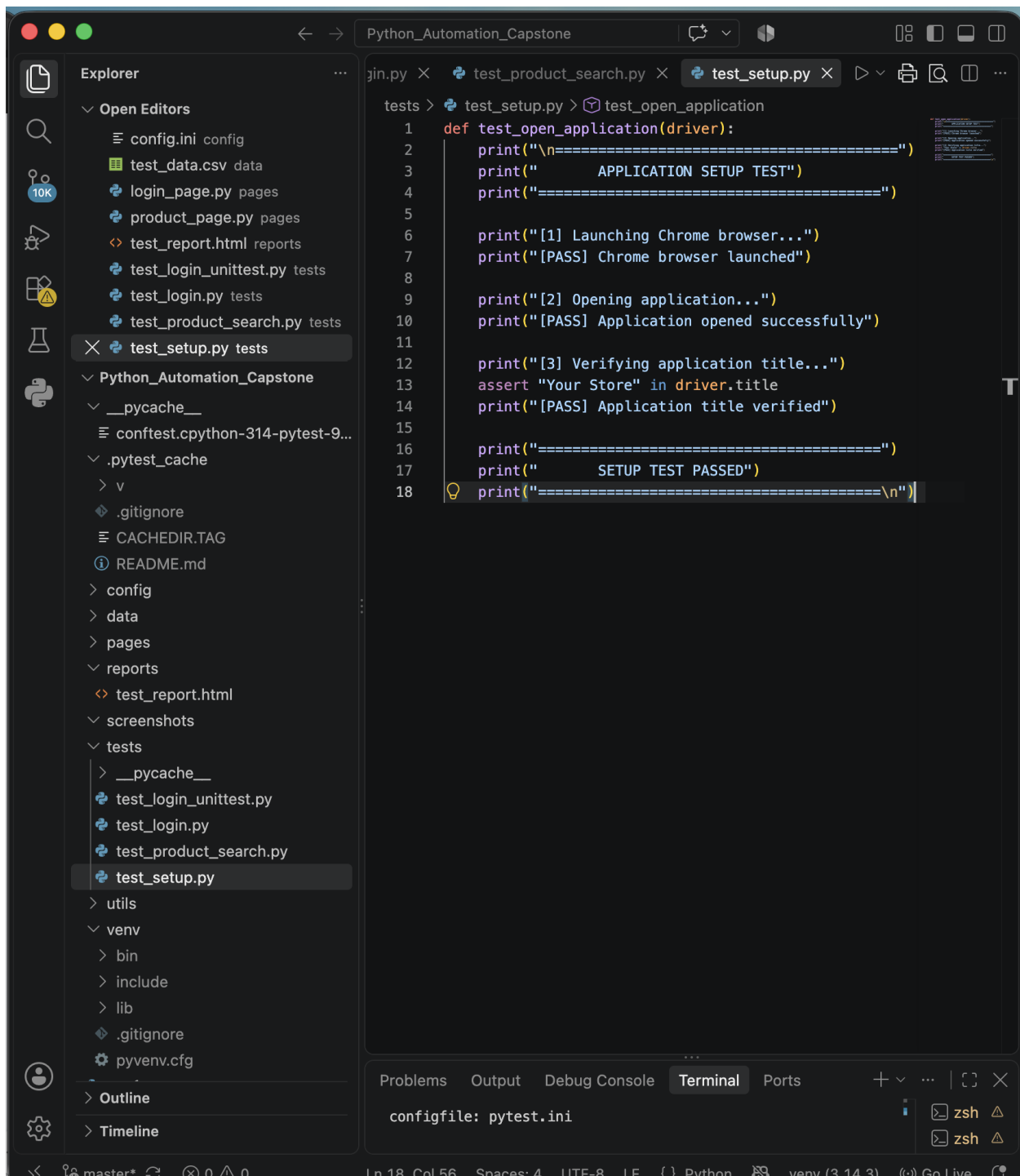
```
print("      APPLICATION SETUP TEST")
print("=====")

print("[1] Launching Chrome browser...")
print("[PASS] Chrome browser launched")

print("[2] Opening application...")
print("[PASS] Application opened successfully")

print("[3] Verifying application title...")
assert "Your Store" in driver.title
print("[PASS] Application title verified")

print("=====")
print("      SETUP TEST PASSED")
print("=====\\n")
```



Folder:- utils

Files:- config_reader.py

```
import configparser
import os
```

```
class ConfigReader:

    def __init__(self):
        self.config = configparser.ConfigParser()

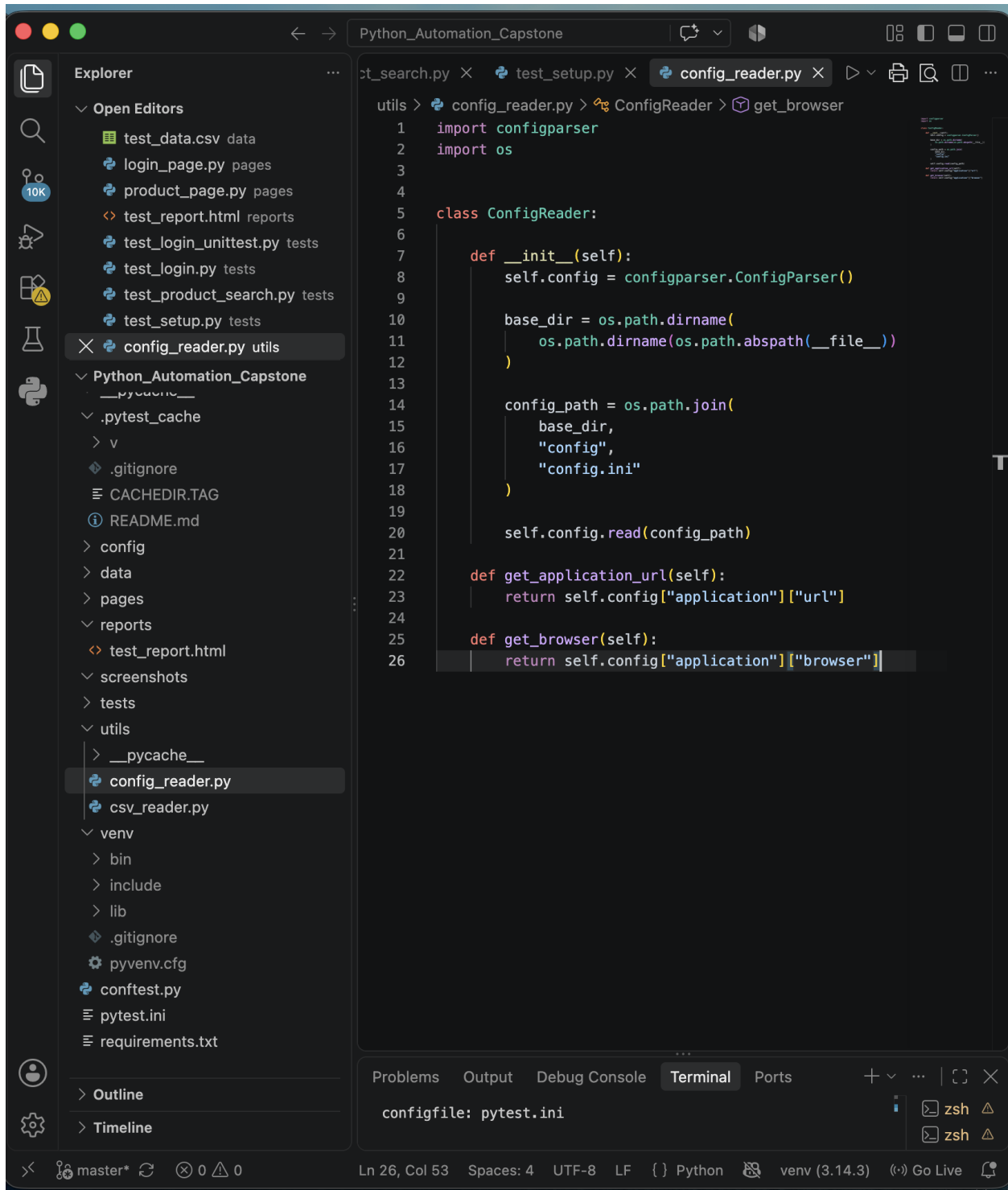
        base_dir = os.path.dirname(
            os.path.dirname(os.path.abspath(__file__)))
        )

        config_path = os.path.join(
            base_dir,
            "config",
            "config.ini"
        )

        self.config.read(config_path)

    def get_application_url(self):
        return self.config["application"]["url"]

    def get_browser(self):
        return self.config["application"]["browser"]
```



Files:- csv_reader.py

```
import csv
import os
```

```

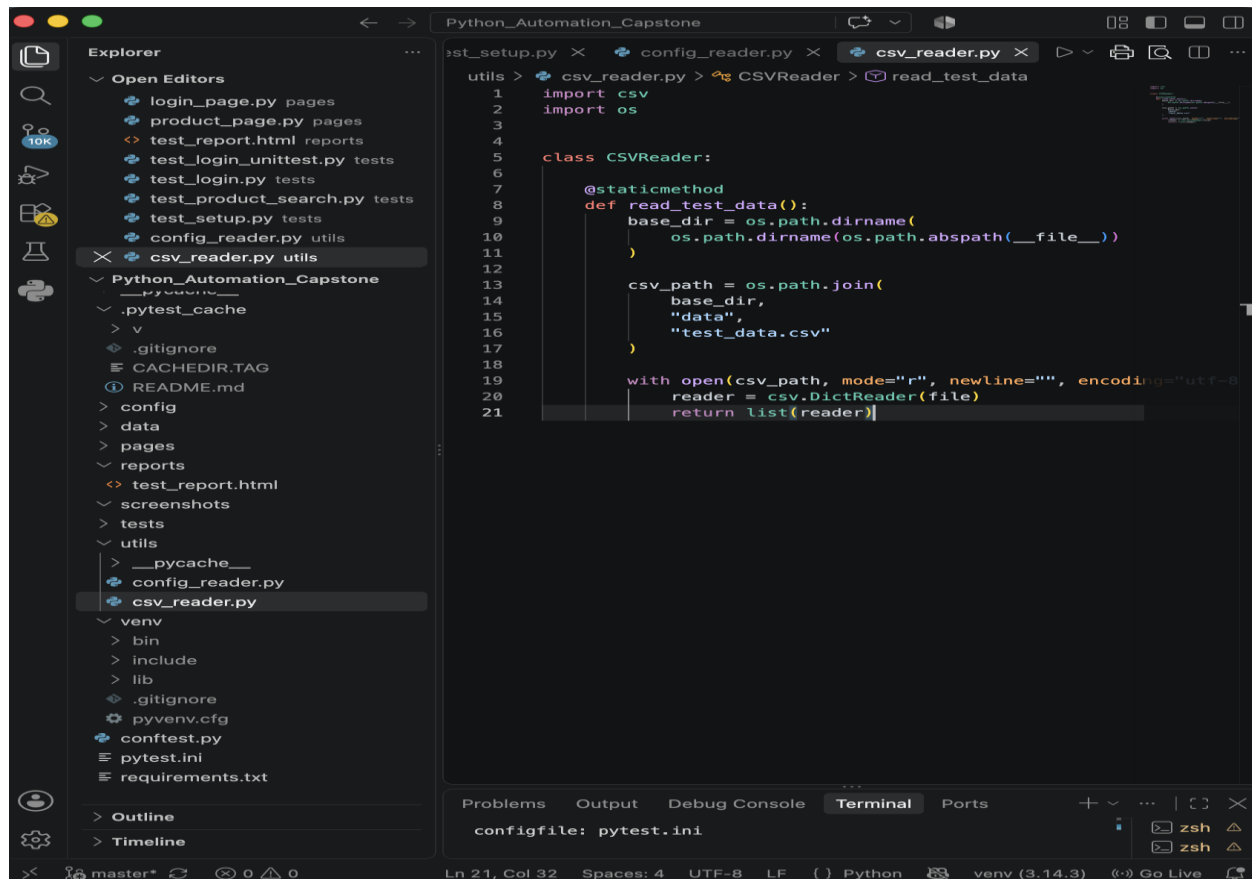
class CSVReader:

    @staticmethod
    def read_test_data():
        base_dir = os.path.dirname(
            os.path.dirname(os.path.abspath(__file__)))

        csv_path = os.path.join(
            base_dir,
            "data",
            "test_data.csv"
        )

        with open(csv_path, mode="r", newline="", encoding="utf-8") as file:
            reader = csv.DictReader(file)
            return list(reader)

```



File:- [confest.py](#)

```

import os

import pytest
from selenium import webdriver

from utils.config_reader import ConfigReader


@pytest.fixture
def driver():
    config = ConfigReader()

    browser = config.get_browser()
    url = config.get_application_url()

    if browser.lower() == "chrome":
        driver = webdriver.Chrome()
    else:
        raise ValueError(f"Unsupported browser: {browser}")

    driver.maximize_window()
    driver.get(url)

    yield driver

    driver.quit()


@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()

    if report.when == "call" and report.failed:
        driver = item.funcargs.get("driver")

        if driver:
            screenshots_dir = os.path.join(
                os.path.dirname(os.path.abspath(__file__)),
                "screenshots"
            )

```

```

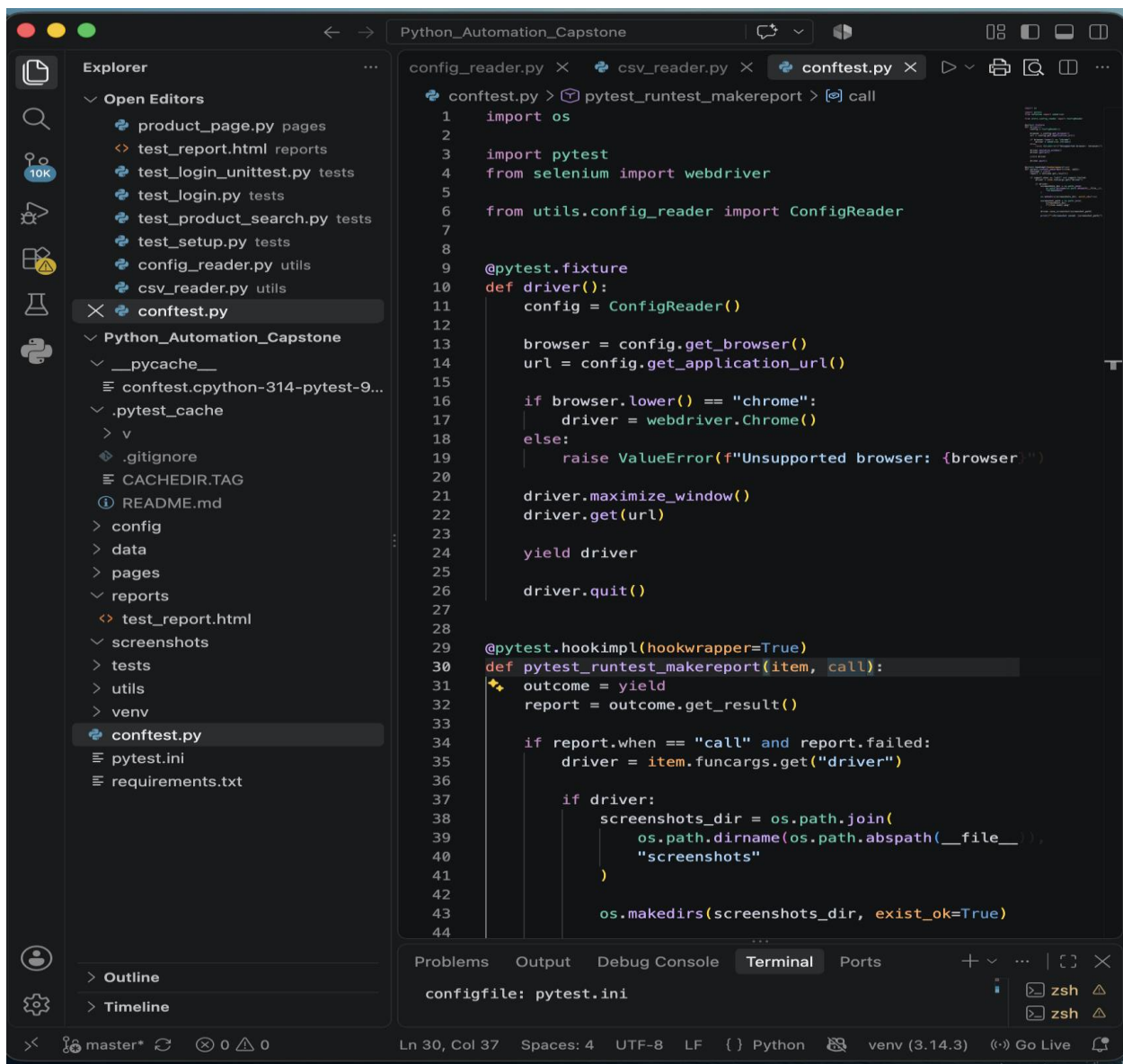
os.makedirs(screenshots_dir, exist_ok=True)

screenshot_path = os.path.join(
    screenshots_dir,
    f"{item.name}.png"
)

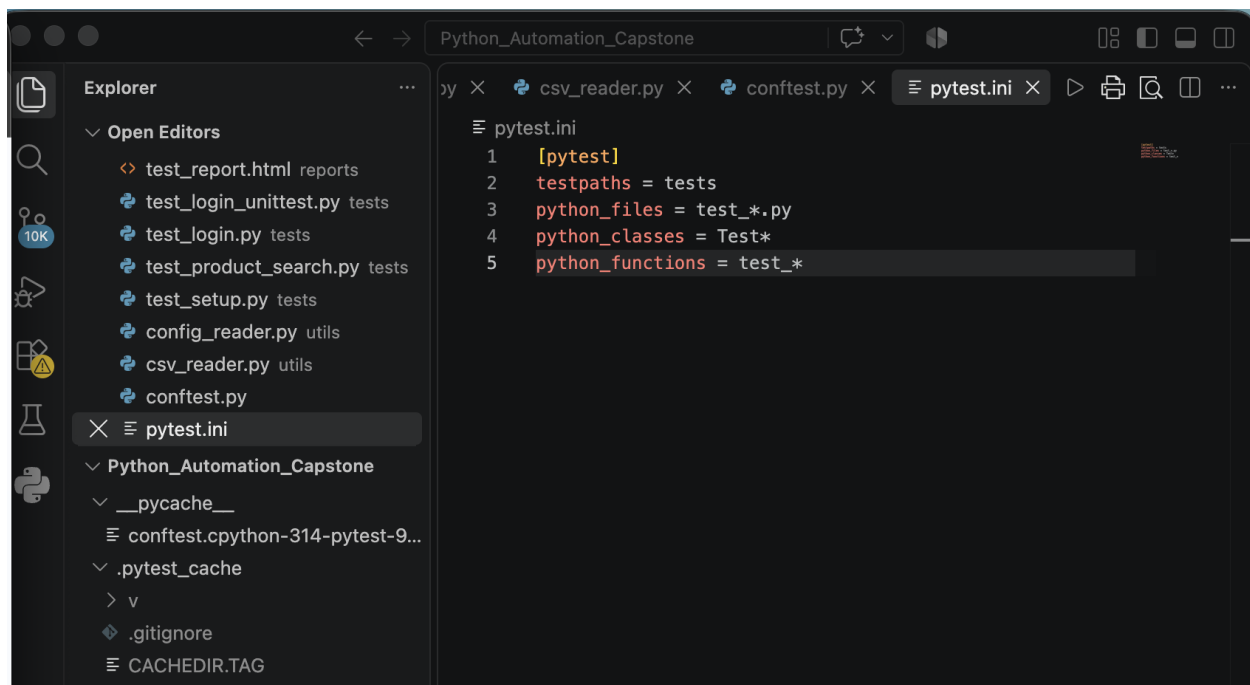
driver.save_screenshot(screenshot_path)

print(f"\nScreenshot saved: {screenshot_path}")

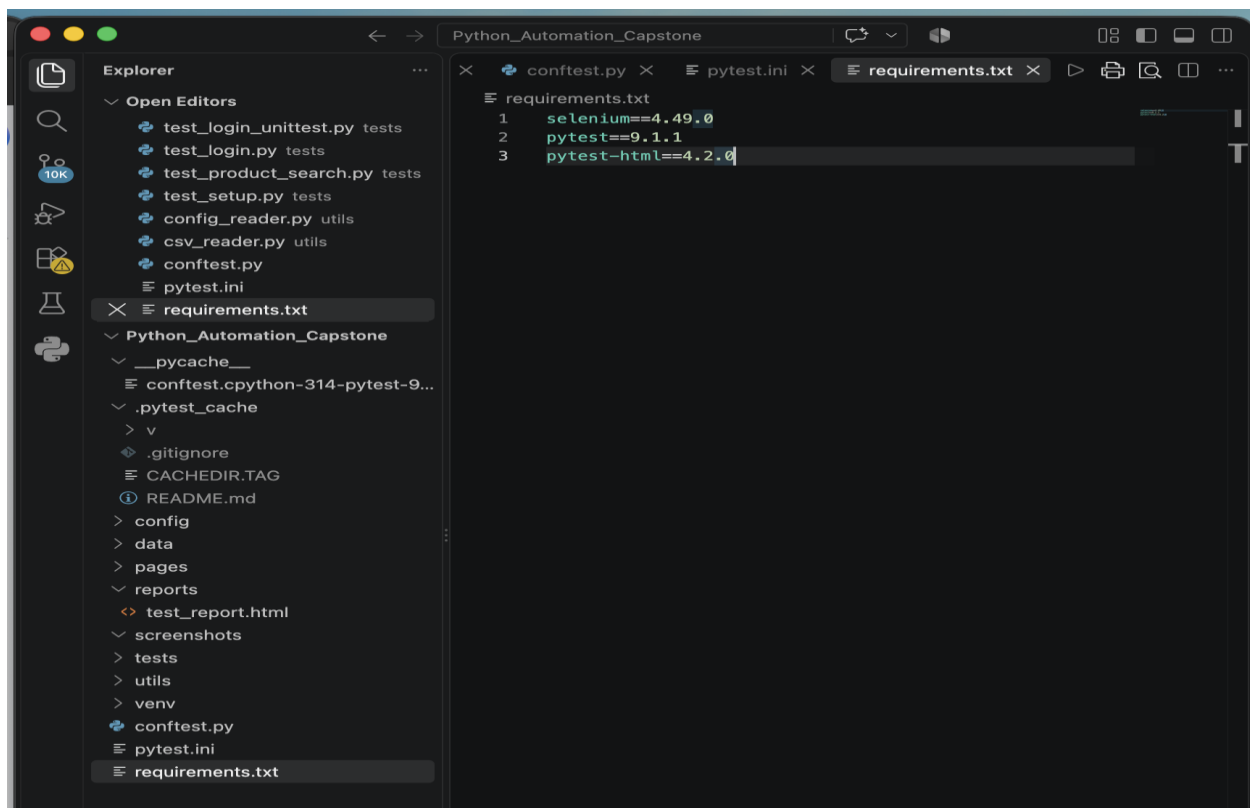
```



Files:- pytest.ini



Files:- requirements.txt



The screenshot shows the VS Code interface with the 'requirements.txt' file open in the Explorer. The file contains the following dependencies:

```
1 selenium==4.9.0
2 pytest==9.1.1
3 pytest-html==4.2.0
```

The Terminal window displays the output of the test session. The tests are categorized into 'LOGIN AUTOMATION TEST' and 'PRODUCT SEARCH AUTOMATION TEST'. The 'LOGIN AUTOMATION TEST' includes a 'test_valid_login' test, which is marked as 'PASSED'. The 'PRODUCT SEARCH AUTOMATION TEST' includes a 'test_product_search' test, which is also marked as 'PASSED'. The output shows the following steps for the login test:

```
[1] Reading login data from CSV...
[PASS] CSV test data loaded successfully
[2] Opening login Page...
[PASS] Login page opened
[3] Entering email and password...
[PASS] Login credentials submitted
[4] Verifying login...
[PASS] Login successful
=====
LOGIN TEST PASSED
=====
```

The output for the product search test is as follows:

```
[1] Reading product data from CSV...
[PASS] Product loaded: MacBook
[2] Opening application...
[PASS] Application opened
=====
PRODUCT SEARCH AUTOMATION TEST
=====
```

The screenshot shows the VS Code interface with the 'requirements.txt' file open in the Explorer. The file contains the following dependencies:

```
1 selenium==4.9.0
2 pytest==9.1.1
3 pytest-html==4.2.0
```

The Terminal window displays the output of the test session. The tests are categorized into 'APPLICATION SETUP TEST' and 'PRODUCT SEARCH AUTOMATION TEST'. The 'APPLICATION SETUP TEST' includes a 'test_open_application' test, which is marked as 'PASSED'. The 'PRODUCT SEARCH AUTOMATION TEST' includes a 'test_product_search' test, which is also marked as 'PASSED'. The output shows the following steps for the application setup test:

```
[1] Launching Chrome browser...
[PASS] Chrome browser launched
[2] Opening application...
[PASS] Application opened successfully
[3] Verifying application title...
[PASS] Application title verified
=====
SETUP TEST PASSED
=====
```

The output for the product search test is as follows:

```
[1] Reading product data from CSV...
[PASS] Product loaded: MacBook
[2] Opening application...
[PASS] Application opened
[3] Searching for product...
[PASS] Search performed for: MacBook
[4] Verifying search results...
[PASS] Product found in search results
=====
PRODUCT SEARCH TEST PASSED
=====
```

The final output shows the test summary:

```
- Generated html report: file:///Users/amitraj/Desktop/Python_Automation_Capstone/reports/test_report.html -
- (venv) amitraj@M1TS-MacBook-Air Python_Automation_Capstone %
- History restored
```